

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

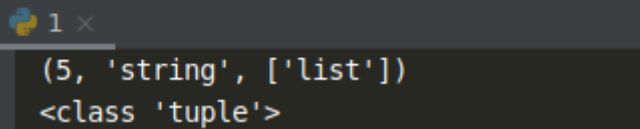
Кортеж – Tuple	2
Словарь – Dictionary	6
Методы словарей.....	8
Метод update	8
Метод values	9
Метод keys	9
Метод clear.....	9
Метод copy.....	10
Метод items.....	10
Метод get	10
Множество – Set	12
Методы множеств	13
Метод add	13
Метод clear.....	13
Метод update	14
Метод remove	14
Метод discard.....	14
Метод union.....	14
Метод intersection	15
Тип None.....	16

Кортеж – Tuple

Главное отличие кортежа от списков довольно простое – если список мы можем изменить, добавив или удалив оттуда нужные нам элементы, то при попытке внести изменения в созданный кортеж, мы получим ошибку.

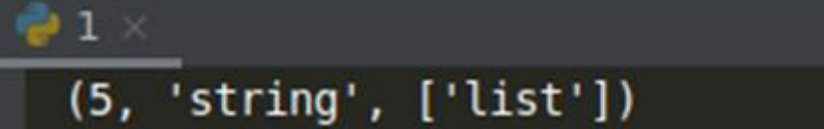
Кортежи занимают меньше памяти, поскольку не содержат такое разнообразие методов, как списки, а доступ к ним осуществляется быстрее. Так же, как и списки, кортежи поддерживают смешанные типы данных. Создать кортеж можно парой круглых скобок:

```
main_tuple = () # пустой кортеж
main_tuple_on = (5, 'string', ['list'])
print(main_tuple_on)
print(type(main_tuple_on))
```



Другой вариант создания кортежа – перечислить его содержимое через запятую после символа равенства:

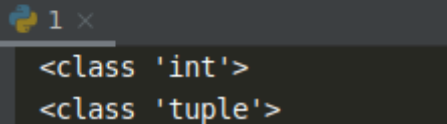
```
my_tuple = 5, 'string', ['list']
print(my_tuple)
```



Подобный способ рабочий, но использовать его **не рекомендуется** ввиду его неочевидности.

Есть одна особенность создания кортежа – если кортеж состоит только из одного элемента, то после него **ОБЯЗАТЕЛЬНО** должна стоять запятая. В противном случае, такая запись не будет интерпретироваться как кортеж.

```
main_tuple_one = (5)
main_tuple_two = (5,)
print(type(main_tuple_one))
print(type(main_tuple_two))
```



К элементам кортежа можно обращаться, используя индексы и срезы:

```
main_tuple = (5, 'string', ['list'], 'string_two')
print(main_tuple[0])
print(main_tuple[-1])
print(main_tuple[1:-1])
```

```
1 x
5
string_two
('string', ['list'])
```

Приём конкатенации также работает в кортежах. А с помощью метода `join` мы можем объединить содержимое кортежа в строку.

```
main_tuple = ("What's", "your", "name?")
main_tuple_one = ("My", 'name', "is", "Alexander")
print(main_tuple + main_tuple_one)
print(' '.join(main_tuple + main_tuple_one))
```

```
1 x
("What's", 'your', 'name?', 'My', 'name', 'is', 'Alexander')
What's your name? My name is Alexander
```

Вы можете преобразовать список в кортеж, используя функцию `tuple()`.

```
main_list = ["GET", "POST", "PUT", "HEAD"]
main_tuple = tuple(main_list)
print(main_tuple)
```

```
1 x
('GET', 'POST', 'PUT', 'HEAD')
```

Кортеж может включать в себя другие кортежи

```
main_tuple = (("woodpecker", "crow"), "sparrow")
print(len(main_tuple))
```

```
1 x
2
```

Поскольку кортежи не изменяемы, то удалить отдельный элемент нельзя. Чтобы удалить весь кортеж, можно воспользоваться инструкцией `del`.

```
main_tuple = ("woodpecker", "crow", "sparrow")
del main_tuple
print(main_tuple)
```

1 ×

```
Traceback (most recent call last):
  File "/root/PycharmProjects/education/1.py", line 3, in <module>
    print(main_tuple)
NameError: name 'main_tuple' is not defined
```

Но есть один нюанс – несмотря на то, что кортеж нельзя изменить после создания, остаётся возможность изменить его вложенные элементы, если они относятся к изменяемому типу. Для этого сначала мы указываем индекс элемента, и затем индекс уже вложенного элемента, который мы хотим изменить, и присваиваем ему новое значение.

```
main_tuple = ("матрёшка", "балалайка", "шапка-ушанка")
main_tuple[0][1] = 'баян'
print(main_tuple)
```

1 ×

```
('матрёшка', 'баян', 'шапка-ушанка')
```

К кортежам можно применять уже знакомые вам функции `min`, `max`, и метод `count`.

```
main_tuple = ('Париж', 'Лондон', 'Москва', 'Париж', 'Осло')
print(main_tuple.count('Париж'))
print(min(main_tuple))
print(max(main_tuple))
```

1 ×

```
2
Лондон
Париж
```

Также можно умножать кортежи, а для числовых значений, использовать функцию `sum`.

```
main_tuple = (5, 25, -6, 48, 858)
print(sum(main_tuple))
print(main_tuple*2)
```

1 ×

```
930
(5, 25, -6, 48, 858, 5, 25, -6, 48, 858)
```


Для получения индекса элемента используется аналогичный списковому метод **index**.

```
main_tuple = (6, 'dog', 'cat', 45)
print(main_tuple.index('cat'))
```

```
1 x
```

```
2
```

Кортежи можно отсортировать, если их элементы одного типа. Функция **sorted** **возвращает отсортированный список**. Если нам нужно получить именно кортеж, то для перевода списка в кортеж используем **tuple()**.

```
main_tuple = ('cow', 'dog', 'cat')
print(sorted(main_tuple))
print(tuple(sorted(main_tuple)))
```

```
1 x
```

```
['cat', 'cow', 'dog']
('cat', 'cow', 'dog')
```

Получить из кортежа список без сортировки можно просто применив функцию **list()**.

```
main_tuple = ('cow', 'dog', 'cat')
print(list(main_tuple))
```

```
1 x
```

```
['cow', 'dog', 'cat']
```

Словарь – Dictionary

Одним из наиболее используемых типов данных в python является словарь.

Словарь в Python – это неупорядоченный набор данных произвольного типа с доступом по ключу (начиная с версии python 3.8 порядок элементов в словаре остаётся неизменным). Элементами такой коллекции выступают **пары объектов**, каждая из которых включает в себя **ключ** и **значение**.

Обратите внимание: ключи в словаре должны быть уникальными. Если вы добавите в словарь пару, ключ которой уже есть в словаре, старая пара будет заменена новой.

Создать словарь можно парой фигурных скобок.

```
wordbook = {}  
print(type(wordbook))
```

```
1 x  
<class 'dict'>
```

Значениями словаря могут быть любые типы данных, но ключи должны иметь неизменяемый тип данных, такой как **строки**, **числа** или **кортежи**. Сами словари являются изменяемым типом данных.

```
wordbook_1 = {1: "one", 2: "two", 3: "three"} # ключ:значение  
print(wordbook_1)  
wordbook_2 = {1: 1, 2: "two", 3: [3]} # смешанные типы данных значений  
print(wordbook_2)
```

```
1 x  
{1: 'one', 2: 'two', 3: 'three'}  
{1: 1, 2: 'two', 3: [3]}
```

Словарь также можно создать с помощью функции **dict()**. Синтаксис следующий – ключ пишется без кавычек, причём ключом может быть только строка, и значение присваивается после знака равно.

```
wordbook = dict(Frolov='chief', Panin='admin', Zukov='master')  
print(wordbook)
```

```
1 x  
{'Frolov': 'chief', 'Panin': 'admin', 'Zukov': 'master'}
```

Для того чтобы внести в коллекцию новую пару объектов, необходимо указать новый ключ в квадратных скобках, а также соответствующее ему значение.

```
wordbook = dict(Frolov='chief', Panin='admin', Zukov='master')
wordbook['Kozlov'] = 'working'
print(wordbook)
```

```
1 x
{'Frolov': 'chief', 'Panin': 'admin', 'Zukov': 'master', 'Kozlov': 'working'}
```

Таким образом можно заполнить пустой словарь.

```
wordbook = {}
wordbook[1] = "right"
wordbook[2] = "left"
wordbook[3] = "forward"
wordbook[4] = "backward"
print(wordbook)
```

```
1 x
{1: 'right', 2: 'left', 3: 'forward', 4: 'backward'}
```

Удалить пару **ключ:значение** можно с помощью инструкции **del**, указав в квадратных скобках только ключ.

```
wordbook = {'Игра 1': "Doom", 'Игра 2': "Darkwood"}
del wordbook['Игра 1']
print(wordbook)
```

```
1 x
{'Игра 2': 'Darkwood'}
```

Проверка наличия **ключа** в словаре производится с помощью оператора **in**.

```
wordbook = {'Игра 1': "Doom", 'Игра 2': "Darkwood"}
print('Игра2' in wordbook)
print('Игра 2' in wordbook)
```

```
1 x
False
True
```


Для доступа к значению словаря в качестве индекса указывается ключ.

```
wordbook = {'Иванов': '20000 руб', 'Петров': '50200 руб'}
print(wordbook['Петров'])
```

```
1 x
50200 руб
```

Мы так же можем создать из словаря строку следующим образом:

```
wordbook = {'духи': 'Шанель', 'одеколон': 'Ромео'}
print(str(wordbook))
print(type(str(wordbook)))
```

```
1 x
{'духи': 'Шанель', 'одеколон': 'Ромео'}
<class 'str'>
```

Узнать длину словаря (количество пар ключ:значение), можно стандартной функцией `len`.

```
wordbook = {'Шерлок': 'Холмс', 'Джон': 'Ватсон'}
print(len(wordbook))
```

```
1 x
2
```

Методы словарей

Метод update

Мы можем при необходимости обновить данные одного словаря, данными другого, которые будут записаны в конец первого словаря. Делается это с помощью метода `update()`.

```
wordbook_1 = {1: "right", 2: "left"}
wordbook_2 = {3: "forward", 4: "backward"}
wordbook_1.update(wordbook_2)
print(wordbook_1)
```

```
1 x
{1: 'right', 2: 'left', 3: 'forward', 4: 'backward'}
```

Метод values

Данный метод возвращает список всех значений доступных из словаря.

```
wordbook = {'Высота': 'Метры', 'Вес': 'Килограммы'}  
print(wordbook.values())  
  
1 ×  
dict_values(['Метры', 'Килограммы'])
```

Этот метод вернёт нам объект типа `dict_values` – значения словаря, но мы можем привести его к обычному списку, воспользовавшись функцией [list](#).

```
wordbook = {'Высота': 'Метры', 'Вес': 'Килограммы'}  
print(list(wordbook.values()))  
  
1 ×  
['Метры', 'Килограммы']
```

Метод keys

Этот метод возвращает список всех ключей в словаря.

```
wordbook = {'Да': 'Yes', 'Нет': 'No'}  
wordbook2 = wordbook.keys()  
print(wordbook2)  
  
1 ×  
dict_keys(['Да', 'Нет'])
```

Метод clear

Метод [clear](#) удаляет все элементы словаря.

```
wordbook = {'Высота': 'Метры', 'Вес': 'Килограммы'}  
wordbook.clear()  
print(wordbook)  
  
1 ×  
{}
```

Метод copy

Создаёт копию словаря.

```
wordbook = {'Хищник': 'Волк', 'Жертва': 'Олень'}  
wordbook2 = wordbook.copy()  
print(wordbook2)
```

1 ×

```
{'Хищник': 'Волк', 'Жертва': 'Олень'}
```

Метод items

Этот метод возвращает список пар в виде кортежей.

```
wordbook = {'Да': 'Yes', 'Нет': 'No'}  
wordbook2 = list(wordbook.items())  
print(wordbook2)
```

1 ×

```
[('Да', 'Yes'), ('Нет', 'No')]
```

Постарайтесь запомнить этот метод, мы часто будем использовать его в циклах.

Метод get

Данный метод похож на использование индексов, и возвращает значение из словаря по ключу.

```
wordbook = {'Да': 'Yes', 'Нет': 'No', 'Неизвестный': 'unknown'}  
print(wordbook.get('Неизвестный'))
```

1 ×

```
unknown
```

Главное отличие этого метода от получения объекта по ключу в том, что он не создаст исключение, если ключ не будет найден.

Метод setdefault

Метод setdefault довольно работает довольно хитро: он принимает 2 аргумента – ключ и значение. Если значение **соответствует** ключу словаря, мы получим это же значение. Если же значение **не соответствует**, он вернёт тот объект, который находится по ключу

```
1 my_dict = {1: 'a', 2: 'b', 3: 'c'}
2
3 result = my_dict.setdefault(2, 'Б')
4 print(result, type(result), my_dict)
5
6 second_try = my_dict.setdefault(4, 'd')
7 print(second_try, type(second_try), my_dict)
```

Run: 1 x

```
b <class 'str'> {1: 'a', 2: 'b', 3: 'c'} 1
d <class 'str'> {1: 'a', 2: 'b', 3: 'c', 4: 'd'} 2
```

После объявления словаря my_dict мы попытались переопределить значение ключа 2 новой строкой – 'Б', но из-за того, что этому ключу уже соответствует строка 'b', мы не смогли её изменить. Зато, попытавшись добавить значение 'd' по ключу 4 всё прошло успешно – ключ добавился и словарь обновился новой парой.

Множество – Set

Множество (set) — это **неупорядоченный** набор **уникальных элементов**. Элементы множества изменить нельзя, поэтому множество не может содержать изменяемый элемент, такой как список или словарь.

Само множество относится к изменяемому типу данных. Создать множество можно фигурными скобками, с перечислением элементов через запятую – в этом оно очень похоже на словарь.

```
main_set = {'ручей', 'река', 'море', 'река'}
print(main_set)
```

1 ×

```
{'ручей', 'море', 'река'}
<class 'set'>
```

Чтобы создать пустое множество в Python, мы должны использовать функцию **set()** без передачи какого-либо значения.

```
main_set = set()
print(type(main_set))
```

1 ×

```
<class 'set'>
```

Функция **set()** принимает только 1 аргумент. Каждый элемент может входить во множество только один раз. Таким способом мы преобразовали строку в сет. Аналогично работает для списков и кортежей.

```
main_set = set('подарок')
print(main_set)
```

1 ×

```
{'п', 'к', 'о', 'р', 'а', 'д'}
```

Проверить вхождение элемента в сет можно с помощью оператора **in**.

```
main_set = {'Атос', 'Партос', 'Арамис'}
print('Партос' in main_set)
print('Ришелье' in main_set)
```

1 ×

```
True
False
```

Или, наоборот, убедиться, что элемента нет в списке, можно применив оператор **not in**.

```
main_set = {'Атос', 'Партос', 'Арамис'}
print('Партос' not in main_set)
print('Ришелье' not in main_set)
```

1 ×

False

True

Узнать точное количество элементов, входящих в состав множества, можно стандартной функцией **len**.

```
main_set = {48, -50, 0, 10, 0, 12, 48}
print(len(main_set))
```

1 ×

5

Методы множеств

Метод add

Внести новые значения в сет можно методом **add()**.

```
main_set = {48, -50, 0, 10, 12}
main_set.add('Добавь меня')
print(main_set)
```

1 ×

{0, 'Добавь меня', 10, 12, -50, 48}

Метод clear

Методом **clear()** удаляются все элементы множества.

```
main_set = {'макака', 'шимпанзе', 'гамадрил'}
main_set.clear()
print(main_set)
```

1 ×

set()

Метод update

Для добавления одного множества к другому, используется метод `update()`.

```
main_set_1 = {'макака', 'шимпанзе', 'гамадрил'}
main_set_2 = {'павиан', 'мадрил'}
main_set_1.update(main_set_2)

1 x
{'павиан', 'шимпанзе', 'мадрил', 'макака', 'гамадрил'}
```

Метод remove

Этим методом можно удалить любой элемент множества.

```
main_set = {48, -50, 0, 10, 12}
main_set.remove(0)
print(main_set)

1 x
{10, 12, -50, 48}
```

Метод discard

Метод `discard()` удаляет указанный элемент в скобках. Метод практически аналогичен методу `remove()`. Отличается тем, что при попытке удаления несуществующего элемента методом `discard()` никакой ошибки не будет. А при удалении с помощью метода `remove()`, возникнет ошибка.

```
main_set = {'остров', 'полуостров', 'гора'}
main_set.discard('гора')
print(main_set)

1 x
{'полуостров', 'остров'}
```

Метод union

Объединяет элементы нескольких множеств в одно, удаляя все дубли.

```
main_set_1 = {'остров', 'полуостров', 'гора'}
main_set_2 = {'вулкан', 'атолл'}
print(main_set_1.union(main_set_2))

1 x
{'остров', 'полуостров', 'гора', 'атолл', 'вулкан'}
```

Также есть аналогичный оператор вертикальная черта |

```
main_set_1 = {'остров', 'полуостров', 'гора'}
main_set_2 = {'вулкан', 'атолл'}
print(main_set_1 | main_set_2)
```

1 ×

```
{'вулкан', 'остров', 'полуостров', 'атолл', 'гора'}
```

Метод intersection

Метод `intersection()` используется для пересечение сетов – операции получения набора элементов, которые являются общими для обоих множеств.

```
main_set_1 = {'остров', 'полуостров', 'гора'}
main_set_2 = {'вулкан', 'атолл', 'риф', 'остров'}
print(main_set_1.intersection(main_set_2))
```

1 ×

```
{'остров'}
```

Аналогичного эффекта можно добиться, применив оператор амперсанд `&`.

```
main_set_1 = {'остров', 'полуостров', 'гора'}
main_set_2 = {'вулкан', 'атолл', 'риф', 'остров'}
print(main_set_1 & main_set_2)
```

1 ×

```
{'остров'}
```

Во множестве нельзя обратиться к элементам по индексу, так как коллекция является неупорядоченной.

Один из наиболее часто встречающихся способов применения множеств – удаление дубликатов из набора данных. Приведя любой перечисляемый тип ко множеству, мы избавимся от повторяющихся элементов в нём.

Тип None

Специальный тип **None** означает **ничего** - используется для обозначения «пусто» или «здесь нет значения».

```
f = None
print(type(f))

1 x
<class 'NoneType'>
```

Может применяться вместе с условными операторами - «если что-то не **None**, можно продолжать работу программы».

Присвоение значения **None** переменной, является одним из способов сброса ее в исходное, пустое состояние.

```
f = 'Что-то'
f = None
print(f)

1 x
None
```